



[BIT703 Web Technologies](#) / [Contents](#) / Topic 10: Server-side web application development with procedural and object-oriented database programming / PHP OOP

Classes and objects 60m

Let's define a class and an object.

- A class can be defined as a programmer-defined data type, which includes local functions as well as local data. A class is a template for making many instances of the same class of object.
- An object can be defined as an individual instance of the data structure defined by a class. One can define a class once and then make many objects that belong to it. Objects are also known as instances.

To put it simply, a class can be thought of as a template for multiple instances of the same type of object, while an object can be thought of as an individual instance that is defined by the class. An object will have both states (attributes) and behaviours (methods).

Take, for example, your bank account. A bank account has an account number and a balance, which are the states (attributes). A bank account also has methods that let you withdraw and deposit money, which are the behaviours. So, a bank account is an object. Do you have more than one bank account? Most people have multiple accounts. For example, you may have separate accounts for savings, everyday spending and bills. Since a class is a template for objects, a bank account class can be used to create multiple bank account objects, each with its own account number and balance.

[Skip to main content](#)

OOP classes

We know that a class is a template for objects, and an object is an instance of a class. But how do you define a class?

General form

The general form for defining a new class in PHP is:



```
1  <?php
2  class myClass {
3      //attributes of the class
4      var $var1;
5      var $var2 = "constant string";
6      //methods of the class
7      function myfunc ($arg1, $arg2) {
8          [...]
9      }
10     [...]
11 }
12 ?>
```



Let's break down the class syntax above.

- A class, myClass, is defined using the `class` keyword.
- The keyword `class` is followed by the name of the class that you want to define.
- A set of curly braces `{ }` enclose the variable declarations (attributes) and function definitions (methods).

Skip to main content

- The variable declarations start with the keyword `var` and are followed by a conventional `$` variable name. They may also have an initial assignment to a constant value.
- Function definitions look like standalone PHP functions, but they are associated with the class and will be used to set and access the object data.

To help you understand classes better, let's look at a real-world example. Let's assume that we have a class named `Book`. A book can have attributes like title, price, author and so on. Your class would have:

- variables that hold the values of these attributes, such as `$title` and `$price`
- functions that set out the methods used, such as `getTitle()` and `setTitle()`, which are the methods used to set and get the `$title` attributes.

The example below declares the `Book` class and its attributes and methods that we've just discussed:



[Skip to main content](#)



```
1  <?php
2      class Book {
3          /* Attributes. Variables are declared in the class Book */
4          var $price;
5          var $title;
6
7          /* Methods */
8          function setPrice($par){
9              $this->price = $par;
10         }
11
12         function getPrice(){
13             echo $this->price . "<br/>";
14         }
15
16         function setTitle($par){
17             $this->title = $par;
18         }
19
20         function getTitle(){
21             echo $this->title . " <br/>";
22         }
23     }
24  ?>
```



The variable **\$this** is a special variable, and it refers to the same object (that is, itself). This is only used within the methods of the class. This lets you reference the attributes within the class.

[Skip to main content](#)

Creating objects in PHP

Once you have defined your class, objects can be created. When an individual object is created, that object inherits all the attributes and behaviours from the class.

Returning to the Book class example, if we create an object for a PHP book, it will inherit all the attributes and behaviours that have been defined in the class Book, but it will have different attribute values.

To create an object instance, use the `new` operator. In the example below, we will create book objects. `$computerbk`, `$mathsbk` and `$phpbk` are instances of the class Book:

```
$computerbk = new Book;  
  
$mathsbk = new Book;  
  
$phpbk = new Book;
```

Here we have created three objects. These objects are independent of each other and exist separately.

Next, we will look at how to access methods and process attributes.

Calling member functions

First, let's define a method. A method is a function declared inside a class, typically used to access or modify the data associated with an object.

Once an object is created, you can access the methods that relate to that object by using the `->` operator. `->` is referred to as an arrow operator that is used to access the methods and attributes of a class object.

Methods can access properties; these are the variables defined inside a class. This means that one member function will be able to process properties of related objects.

[Skip to main content](#)

[Skip to main content](#)

The following example shows how to set the title and price for our book objects by using the calling member functions. As you can see, the `->` operator has been used alongside **setTitle** to define the book's title and **setPrice** to define the price of the book.

```
$computerbk->setTitle( "Computer Basics for Dummies" );  
  
$mathsbk ->setTitle( "Advanced Algebra" );  
  
$phpbk ->setTitle( "PHP – A Complete Guide" );  
  
$computerbk ->setPrice( 10 );  
  
$mathsbk ->setPrice( 15 );  
  
$phpbk ->setPrice( 7 );
```

Once you've set the titles and prices, you can call them up using the **getTitle** and **getPrice** methods to get the values that have been set using the code above. The code below demonstrates how to do this:

```
$computerbk ->getTitle();  
  
$mathsbk ->getTitle();  
  
$phpbk ->getTitle();  
  
$computerbk ->getPrice();  
  
$mathsbk ->getPrice();  
  
$phpbk ->getPrice();
```

Over to you!



Skip to main content

— — . . .

☑ Book class

Non-Assessed

We've been following an example about a Book class that has been used to create three Book objects: \$computerbk, \$mathsbk and \$phpbk.

In this activity, you must execute the code for this Book class to get the title and price values for each of these three Book objects, then add an 'author' attribute with its associated methods.

To do this, you must combine the code from above to:

- create the **Book class**
- create the **three Book objects**
- **set** the titles and prices by using the calling member functions
- **get** the titles and prices
- set and get the author names (Author A, Author B, Author C).

What is the output of this Books code?

- ☐ I have successfully completed this task.

Submit



Thoughts on this page?



Skip to main content

